

Task 控制台设计审查文档

实验侧可配置仪表盘 / frontend 组件 / 扩展到真实设备

五层架构 (device/measurement/processor/task/plot) + SignalHub、面板几何与密封 API、Panel Editor、真机接线

2026-06-18

Contents

1	这份文档讲什么	1
1.1	覆盖范围	1
1.2	涉及的源文件	2
1.3	如何重建本文档	3
2	总览: 实验侧的可配置仪表盘	4
2.1	它解决什么问题	4
2.2	设计原则 (贯穿全文)	4
3	五层架构与解耦	7
3.1	五层与数据流	7
3.2	逻辑节点共用一个基类	8
3.3	为什么这样分	8
4	SignalHub: 数据契约	9
4.1	发布侧	9
4.2	消费侧	9
4.3	线程安全与 version 跳刷	9
4.4	约定俗成的信号名	9
5	逻辑节点: 数据生产者	11
5.1	LogicNode 基类	11
5.2	Measurement: 驱动设备采集	11
5.3	Processor: 逐帧/反应式变换 ("func" 层)	12
5.4	Task: 一次性编排	12
5.5	装载读出 = 三个节点自己拼	12
5.6	Judge occupancy 反应式处理器	13
5.7	多节点做 A-B	13
6	读出标定 task 与脉冲编程接口	14
6.1	calibrate-readout task 的完整参数设计	14
6.2	脉冲编程接口对齐 GUI	15

7	面板模型	16
7.1	六种面板类型 (kind)	16
7.2	来源 = 表达式	16
7.3	错误隔离与形状自适应	17
8	面板几何:confocal 模数规则	18
8.1	所有 kind 共享同一块 plot 区域	18
8.2	显示缩放:qt_canvas 三不变量	18
9	卡片与布局	19
9.1	卡片 = 带标题的 Fluent 卡	19
9.2	模数闭合:footer 吸收差额	19
9.3	拖动吸附 + 重力压实	19
10	Setting 弹窗:瘦身后的基本配置	20
10.1	持久化:Setting 的每一项都进 PanelConfig.params	22
10.2	声明式参数:加一个旋钮 = 一行	22
11	两个常驻标签 + 每面板/每节点 Edit 架构	23
11.1	VIEW / LOGIC 解耦	24
11.2	task 运行接管控制台 (confocal 式)	24
11.3	每面板 Edit: 冻结快照模型 (关键设计)	25
11.4	Edit 控件 (详细参考 confocal)	25
11.4.1	plot 面板 Edit (PanelEditor)	25
11.4.2	logic 节点 Edit (LogicNodeEditor)	26
11.5	坐标轴 = 源的参数空间; 缩放/框选回写 + 采集环应用 (confocal 核心耦合)	26
12	通用 Measurement 框架	28
12.1	三角色解耦:Axis × Plan × Reducer	28
12.2	引擎:ScannedMeasurement / ScanResult	28
12.3	声明式单一真相源:MeasurementSpec / ParamDecl	29
12.4	自动发现的测量目录:开放注册表	29
12.4.1	单一真相源建器:build_*_scan	29
12.5	GUI 段:自动参数表单 + 一键 Start	30
12.6	虚拟 == 实机	30
12.7	1D 面板的 (N,2) x-y 渲染	30
13	一键温度:release-recapture 测温	31
13.1	物理:弹道重捕	31
13.2	序列:6 周期双触发,t_off=duration scan slot	31
13.3	GUI 路径 (一键)	32
13.4	API 路径 (同源)	32
13.5	虚拟 trap-off 损失模型 (只在数据源侧)	32
14	命名布局:设计一次,随处载回	33
14.1	状态序列化	33

15	空白收紧	34
16	性能结论 (诚实版)	35
16.1	量化 (demo_console=5 面板, 离屏,40 tick)	35
16.2	评估过的方案 (各附一句根因)	35
16.3	合规提速 + 留给用户的开关.	36
17	frontend 密封 API 契约	37
18	frontend 绘图组件参考	38
18.1	工厂与 live 类	38
18.2	LiveSiteMap	38
18.3	DataFigure(拟合/存图/单位)	38
18.4	新增一种 plot type 的方法 (以 sites 为模板)	38
19	扩展到真实设备	40
19.1	最小接线 (notebook, 自管循环)	40
19.2	用内置逻辑节点拼装载读出	41
19.3	对接 references 里的 Rb87 qCMOS 读出	41
19.4	真机 vs 虚拟的唯一差别	42
20	测试与验收	43
21	公共接口与内部结构索引	44
21.1	对外公共 (notebook/实验室会调)	44
21.2	task_console.py 内部要点.	44
22	已知限制与待决方向	45
22.1	优先级.	46

1

这份文档讲什么

这是一份**独立的设计审查文档**，专讲实验侧的 Task 控制台 (task console) 及其依赖的 frontend 组件。目标是把**设计思路、实现方式、以及如何接到真实设备上**一次讲透，供你审查并决定下一步方向。它不是面向终端用户的操作手册（那部分在 frontend 中文手册的“Task 控制台”与“对外接口完整参考”两章），而是面向**维护者与你本人**的架构说明。

1.1 覆盖范围

- 动机：为什么要做实验侧仪表盘，它在整个系统里的位置。
- 五层架构与解耦：SignalHub / 逻辑节点 (LogicNode) / TaskConsole，以及它们**只通过信号集线器耦合**这一原则。
- 每一层的契约与实现细节，连同关键的“坑”与设计取舍。
- 面板几何的 confocal 模数规则，以及 frontend 的**密封 API 契约** (美术/几何/dpi 由 frontend 拥有)。
- **两个常驻标签 (Monitor + Logic) + 每面板/每节点可关闭 Edit 标签**：Monitor= 拖拽 live 网格 (实时监控)；Logic= 逻辑节点行；节点从 Add Panel 新建；每面板 Edit= 该面板专属的参数 / 全套拟合 / 处理，每节点 Edit= 它自己的参数 + Start/Stop (可关闭，柔和 × 关闭键)。
- **confocal 式竖排的每图 Setting 弹窗** (五段:Source / Display(含 colormap 配色) / Unit / Limits / Panel)，及它如何持久化进 PanelConfig.params；重控件 (plot 参数 / 全套 data_figure 拟合) 在每面板的 Edit 标签。
- **逻辑节点的三类生产节点**：measurement (相机 / 扫描) / processor (反应式变换) / task (一次性编排)，都共用 LogicNode 基类、只经 hub 通信。
- **通用 Measurement 框架**: MeasurementSpec / ParamDecl 单一真相源 → ScannedMeasurement (三角色) → ScannedMeasurementNode → Monitor live 曲线。
- **一键温度 (release-recapture 测温)**：物理、序列、GUI 路径、API 路径与时序。
- **读出标定 task、逐帧判读 processor、脉冲编程接口对齐 GUI**。
- 命名布局保存、**空白收紧、性能结论 (诚实版)**。

- **如何扩展到真实设备** (重点章节), 含与 references 里 Rb87 qCMOS 读出流程的对接。
- 测试覆盖、公共/内部函数索引、已知限制与待决方向。

1.2 涉及的源文件

frontend/task_console.py 控制台主体: TaskConsole、PanelCard、LogicNodeRow、LogicNodeEditor、PanelEditor、PanelConfig、TaskConsoleState、Monitor 网格、Logic 列表、每面板/每节点 Edit 标签、命名布局。

neutral_atom/core/signals.py SignalHub: 线程安全命名信号环。

neutral_atom/operations/logic.py LogicNode 共享基类 + 三类生产节点: CameraMeasurement / ScannedMeasurementNode (measurement)、Processor / OccupancyProcessor (processor)、Task / CalibrateReadoutTask + TaskOutput (task)。

neutral_atom/operations/processor.py ProcessorSpec (反应式 make_node 或一次性 run, 二选一) + ProcessorContext; 目录里的 Judge occupancy / Readout fidelity。

neutral_atom/operations/task.py TaskSpec(build/params/mid_run_key/default_kind/prefix); 目录里的 Calibrate readout。

neutral_atom/operations/measurement.py 通用扫描测量引擎: MeasurementSpec/ParamDecl (声明式单一真相源)、ScanAxis/ShotPlan/PointReducer 三角色、ScannedMeasurement/ScanResult、axis_range_tuple。

neutral_atom/operations/measurement_registry.py 开放注册表 + 自动发现: @measurement 装饰器、register_measurement/unregister_measurement、discovered_specs(readout) (导入 operations/measurements/ 包、按 order+name 装配目录); @processor / @task 注册表同构。

neutral_atom/operations/measurements/ 自动发现的测量目录包: 每个 @measurement 工厂一个模块 (内置 temperature.py / readout_duration.py); 丢一个新文件进来就被自动载入。

neutral_atom/operations/temperature.py release-recapture 测温: build_release_recapture_pulse/ReleaseRecapturePulse (单一真相源建器)。

neutral_atom/subsystems/readout.py exp.readout: temperature/readout_duration_fidelity/build_readout_duration_fidelity (单一真相源建器)/camera_measurement/calibrate_task/measurement_specs/processor_specs/task_specs (GUI 三份目录)。

frontend/live.py 绘图工厂 plot/panel_plot、五个 live 绘图类、FigureSpec 几何、panel_plot_spec、PANEL_MARGINS_PX。

frontend/qt_canvas.py EmbeddedFigureCanvas/panel_canvas: 高 DPI 嵌入封装。

frontend/qt_fluent.py Fluent 控件库 + resolve_fluent_auto_scale + FluentTabWidget。

frontend/data_figure.py DataFigure: 拟合 / 存图 / 单位换算 (每面板 Edit 标签复用)。

task_console.py / task_console.bat 根启动器。

1.3 如何重建本文档

源是同目录的 `task\_console\_design\_zh.texbody`; 运行 `docs/task\_console\_design/build.py` 会重新生成 `assets/` 里的截图并在临时目录里编译, **只把 PDF 拷回本目录** (走 frontend 的密封 PDF 接口 `render\_notes\_pdf`, 不留任何 `.tex/.aux/.log` 等中间产物)。

2

总览: 实验侧的可配置仪表盘

2.1 它解决什么问题

脉冲编辑器 (pulse GUI) 解决的是**实验的输出侧**——怎么定义并下发时序到 FPGA。Task 控制台解决的是**实验的输入/监控侧**——实验跑起来之后, 怎么**实时看**原子装载、计数分布、读出保真度、逐站占据这些量, 并且让这套监控**可配置、可保存、可在不同实验间复用**。

一句话: 它是一个网格仪表盘, 里面摆任意多个**实时面板**, 每个面板用一小段**表达式**接到实验信号上, 整套布局存成一个命名的“任务”, 需要时一键载回。

2.2 设计原则 (贯穿全文)

1. **五层解耦, 只经 hub 通信**。device / measurement / processor / task / plot 五层互不直接引用, 唯一的共享态是 `SignalHub`。这让虚拟实验与真机、单节点与多节点、notebook 与独立窗口都能复用同一套面板。
2. **逻辑节点共享一个基类**。三类生产节点(measurement / processor / task)都继承 `LogicNode`, 共用同一套 worker-loop + 协作取消 + owner-thread 参数队列, 所有节点都可同步驱动 (`step()`, 测试/notebook) 或后台线程 (`start(rate\_hz=)`) ——不是各类节点各写一份循环。
3. **数据来源是表达式, 不是下拉框**。面板不绑定到某个固定信号, 而是跑一行 Python (`value = ...`), 所以“两个实验装载率相减”这种需求天然支持, 且新增信号无需改 GUI。
4. **美术/几何由 frontend 拥有, 对外密封**。面板尺寸只能从有限预设里选, dpi/边距/字号/阴影/缩放规则外部都改不了——这样任意类型/尺寸的面板能严丝合缝拼接, 且在不同 Windows DPI 下不崩。
5. **Setting 走 confocal 竖排, 重控件进 Edit 标签**。Setting 弹窗按 confocal 范式做五段 **Source / Display / Unit / Limits / Panel** (每段一个粗体小标题, 下面每行一个控件, 左 64px 标签 + 右控件), 放使用者每张 shot 都会动的轻量控件: 信号 / 表达式 / 尺寸 / colormap 配色 / 单位切换 / x-y lim auto-manual + 上下限。命令行拟合、relim、Save Fig 的进阶处理搬到**该面板专属的 Edit 标签** (Setting 里 `Edit...` 跳过去); 逻辑节点参数表单 + Start/Stop 则在**该节点专属的 Edit 标签** (Logic tab) ——避免弹窗过载。
6. **参数声明一次, API 与 GUI 同源** (借鉴 confocal 的“caller 签名 = 单一真相源”, 但用**显式声**

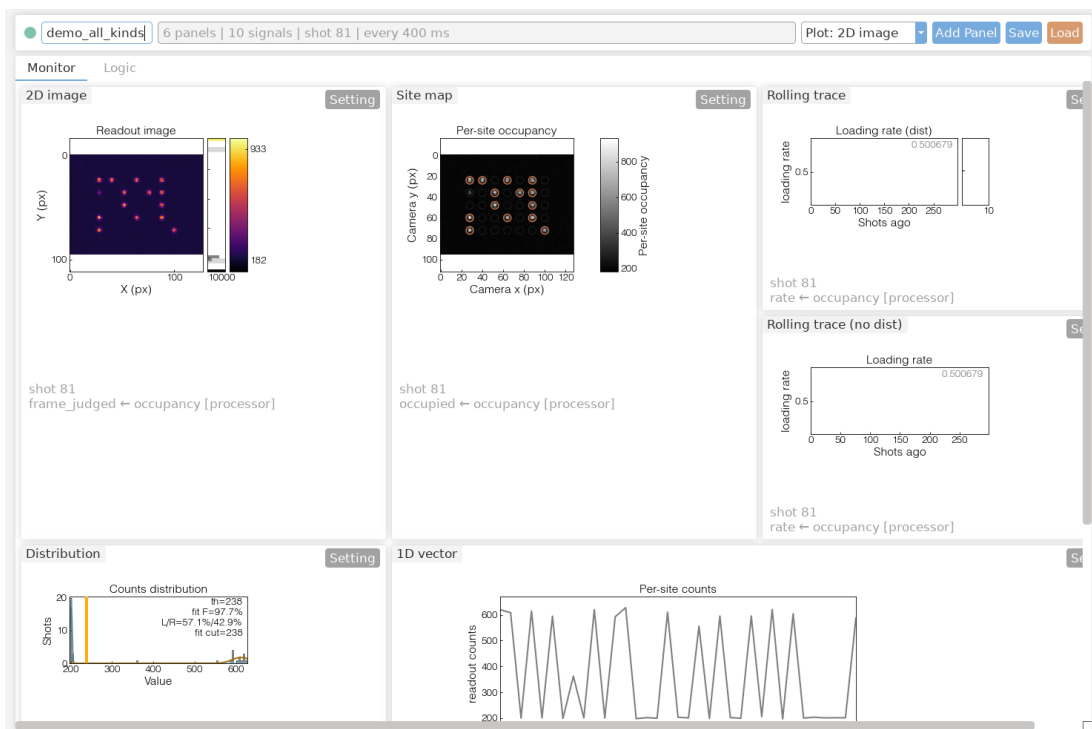


Figure 2.1: 一套自存布局的 Monitor 标签页：左上相机帧、右上占据站点图、计数分布（自动双高斯）、装载率滚动轨迹、底部逐站装载率。控制台**开局空、全停**——这套丰富布局是你自己从 Add Panel 加节点 + 加 Plot 面板拼出、再 Save 成命名布局、一键 Load 回来的。顶栏：任务名 / 摘要 / 面板类型 / Add Panel / Save / Load；**两个常驻标签页**（Monitor= 实时监控网格；Logic= 逻辑节点行）+ 每面板/每节点按需打开的**可关闭 Edit 标签**（面板 Edit 含该面板的处理 / 拟合 / 保存；节点 Edit 含它自己的参数表单 + Start/Stop）。

明而非签名反射/AST)。measurement 用 `MeasurementSpec/ParamDecl`、processor 用 `ProcessorSpec`、task 用 `TaskSpec` 把它的可扫描/可调参数声明一次，API 默认值与 GUI 自动表单都从这份声明派生 → 杜绝两边漂移。

7. **虚拟与真机零修改通用**。逻辑节点发布的信号名 (`frame/counts/occupied/centers/rate`) 就是约定；每个节点只走相机/sequencer/标定契约 (不 import 具体后端、不读仿真真值)。真机沿用同名发布、同一节点，命名布局与测量在两边都能用。

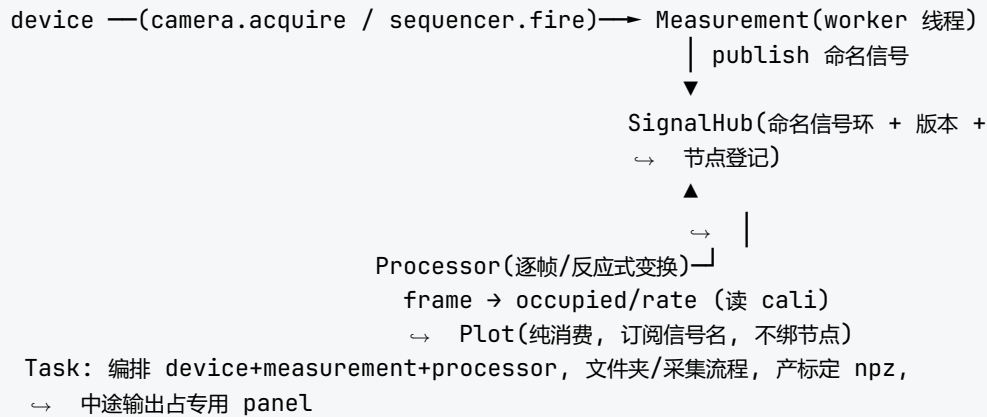
3

五层架构与解耦

3.1 五层与数据流

数据通路分**五层**——device / measurement / processor / task / plot——互相**只通过 SignalHub**耦合：

text



device (设备) 相机 / sequencer。相机带防丢帧 ring-buffer (arm/latest/drain), 缓冲在**相机里**、不在节点层; virtual 相机与真机共用同一 **CameraDevice** 契约, 只把“trigger 来帧”换成仿真帧。

measurement (测量) 一个**驱动设备采集**的逻辑节点: 相机 **CameraMeasurement** 发 frame, 扫描型 **ScannedMeasurementNode** 让有限扫描在 Monitor 里逐点长出 live 曲线、跑完自停。

processor (处理器, “func” 层) 一个**无采集的反应式变换**节点: 消费 hub 信号、republish 派生信号, 如 **OccupancyProcessor** 用与真机读出**相同的 calibration.detect** 契约把 frame 变成 **occupied/counts/rate** 等。

task (任务) 一个**一次性编排**节点, 如 **CalibrateReadoutTask** 产 **TrapCalibration+npz** 产物, 中途把模板帧 + 进度流到一个专用面板。

plot (面板) 即 frontend live 绘图类的复用, **纯消费**: 订阅信号名、按版本水位定时刷新, 不绑任何节点。

3.2 逻辑节点共用一个基类

measurement / processor / task 三类生产节点都是 `LogicNode` (`operations/logic.py`) 的子类, 共享同一套 worker-loop + 协作取消 + owner-thread 参数队列。每类只在“每个 shot 的值从哪来”不同: measurement **从设备采集**、processor **把 hub 信号变换**成派生信号、task **编排多步流程并产出中途输出**。三者只通过 `SignalHub` 耦合:

- 逻辑节点只知道“往 hub publish 命名值”(或反应式 processor 从 hub 读再 republish), 不知道有没有 GUI、有几个面板。
- `TaskConsole` 只知道“从 hub 读最新值/历史”, 不知道数据是虚拟渲染的还是真相机来的。
- 采集循环(真机)可以是逻辑节点的后台线程 (`start(rate\_hz=)`), 也可以是 notebook 里你自己的 `for` 循环——只要每个 shot 调一次 `hub.publish`。

Note

task 的输出不进 hub: 一次性 task 的中途帧/进度写进它自己的 `TaskOutput` 缓冲 (`node.output`), 结果/标定留在实例上 (`node.result/node.calibration`)。hub 只承载 measurement + processor 输出——这样一次性 task 的瞬态帧永不混进 live 信号或被误当读出。控制台把 `node.output` 绑到一张固定面板 (保留键 `\_\_task\_frame\_\_`, 见 Monitor 章), 不经 hub。

3.3 为什么这样分

这套分层直接决定了“扩展到真机”有多容易 (见第 19 章): 把虚拟相机换成真机相机 (只换 `connect()`), 同一套逻辑节点、同一个 GUI 一行都不用改。解耦的代价是多了一层 hub 的拷贝/版本管理, 但换来的是**可测试** (同步 `step()` 驱动, 无线程)、**可移植** (布局 JSON 不含任何机器相关的几何)、**可复用** (多节点同 hub 用前缀做 A-B)。

4

SignalHub: 数据契约

SignalHub (`neutral\_atom/core/signals.py`) 是采集侧与 GUI 之间**唯一的共享态**: 一个线程安全的命名信号环。

4.1 发布侧

`hub.publish(values: Mapping[str, object], *, shot=None) → int`: 把一批命名值写入。每个值被**强制转成 float ndarray (标量变成 0 维) 并拷贝**——所以逻辑节点可以随意复用自己的缓冲区, 不必担心 GUI 读到半写状态。返回新的 version 号。

4.2 消费侧

`latest(name)` 最新值: 0 维返回 float, 否则返回 ndarray 的拷贝; 不存在抛 `KeyError` (错误信息列出可用名)。

`history(name, n=None) → ndarray` 最近若干 shot 堆叠, **形状 (shots, *value_shape)**, axis 0 是 shot; 不足 n 返回现有; 值的形状变过时只保留最近一段同形状连续区。

`names() → list[str]` 当前所有信号名 (排序)。Setting 的信号下拉就是调它。

`snapshot\_latest() → dict` 一致快照 (一次性拿到所有最新值) ——这就是面板表达式求值的命名空间底座。

version / shot 版本计数 / shot 计数。控制台用 version 跳过“没有新数据”的刷新。

4.3 线程安全与 version 跳刷

publish 与读取都在锁内做副本, 所以采集线程和 GUI 线程互不撕裂。控制台的 `\_tick` 先比较 `hub.version`: 没变就直接返回, 不做任何绘图工作——空闲时近乎零开销。

4.4 约定俗成的信号名

逻辑节点发布、且真机建议沿用的一组名字 (沿用它们 = 同一命名布局在虚拟/真机间零修改通用):

信号	形状	含义
<code>frame</code>	<code>(H, W)</code>	最新相机帧
<code>counts</code>	<code>(N_sites,)</code>	逐站 ROI 计数
<code>occupied</code>	<code>(N_sites,)</code>	逐站 0/1 占据
<code>rate</code>	标量	滚动装载率
<code>rate_sites</code>	<code>(N_sites,)</code>	逐站滚动装载率
<code>rate_grid</code>	<code>(rows, cols)</code>	网格形状的逐站率
<code>centers</code>	<code>(N_sites, 2)</code>	标定的站点中心 (相机像素)
<code>thresholds</code>	<code>(N_sites,)</code>	逐站阈值

扫描类节点 (`ScannedMeasurementNode`) 发布 `<x>` / `<y>` / `scan\_done`, 逐站 reducer 另发 `<y>\_sites` (扁平 `(N_sites,)` 向量)。不发布可派生量: 逐站向量的网格视图是 `reshape` 表达式 (`value = <y>\_sites.reshape(ny, nx)`), 而 `shot` 计数器是面板表达式命名空间里的全局变量 (`hub.shot`), 都不再各做一个信号。

5

逻辑节点: 数据生产者

数据从设备进 hub, 靠的是**逻辑节点**。三类生产节点——measurement / processor / task——都继承同一个 `LogicNode` 基类 (`operations/logic.py`), 共享发布循环、协作取消与 owner-thread 参数队列; 它们只在“每个 shot 的值从哪来”上不同。

5.1 LogicNode 基类

`LogicNode` 拥有发布循环; 子类只实现一个 `shot()` (产一个 shot 的 {名: 值}, **空 dict = no-op**: 不发布、shot 计数不前进, 让反应式 processor 只在输入前进时才发)。

step() 同步跑一个 shot 并 publish (加 **prefix**)。测试/notebook 用——无线程、确定性。

start(rate_hz=5.0) 守护线程按速率循环 **step**; 记住 **rate_hz** (暂停后用原速率恢复)。线程里 `shot()` 抛异常会记录 **last_error** + 连续失败计数并 publish **node_error** 让控制台升起横幅, 而非静默杀掉线程 (操作者否则只看到冻结的陈旧数据)。

stop() / **running** 停线程 / 是否在跑。

published_signals() 这个节点 (behind **prefix**) 发布的信号名集合——让控制台把一张面板映射回产它数据的节点, 再暴露那个节点的参数。

acquisition_parameters() / **apply_acquisition_parameters(...)** 数据源自哪些参数可编辑、并安全地接住来自 GUI 线程的改动。运行时采集环是数据源唯一持有者, 改动被**排队**、由采集环在两帧间应用 (详见 Monitor 章的耦合分析)。

prefix 多节点同 hub 时的命名前缀 (如 **b_**) ——这就是 A-B 的基础。

layer / **node_label** / **display_label** 节点属于哪一层 (measurement/processor/task) + 它的短人名 (**camera/detect/calibrate/某曲线名**); 面板 footer 的信号图例用**层名**显示信号来路, 绝不漏 Python 类名。

5.2 Measurement: 驱动设备采集

`Measurement` (`LogicNode` 子类) 每个 shot 从设备采集。连续 vs 扫描只是 **update_mode** (successive shot 怎么累积: **single/replace/roll/average/repeat**), 不是另一个类。

CameraMeasurement 把**原始相机帧**流进 hub (发 `frame`, 多触发时另发 `frame\_0/frame\_1.....`), 不做站点分析。数据源**就是相机**, 所以可编辑的采集参数就是相机自己的 `exposure / region` (ROI 端点) / `frames\_per\_cycle`, 经 `camera.configure` live 下发。backend-agnostic: 真机相机与 `VirtualCamera` 完全同一契约。

ScannedMeasurementNode 把一个**被扫描的测量**包成节点, 让一个有限扫描在 Monitor 里像装载率轨迹那样**长出一条 live 曲线**。每个 `shot()` 前进一个**扫描点** (经 `measurement.measure(value, index)` → 相机/sequencer/标定契约), 并 publish**累积曲线** `\{x\_key: x[:k], y\_key: y[:k], scan\_done, shot\}`, 于是 Monitor 的 1D 面板 (`value = <y\_key> vs <x\_key>`) 逐点填满。**有限扫描语义**: 发布完最后一点后节点自置 stop, 后台 `start()` 线程自行退出; `finished` 报告完成, `run\_to\_completion()` 同步跑完剩余点 (测试/headless)。它只碰 `measure` 与 `hub.publish`, 零 import 具体后端、零读仿真真值 → 自动受 `tests/test\_virtual\_equals\_real\_contract.py` 守护。详见第 12 章。

plot=True / plot=False 分流。notebook 里直接 API 调一个 `measurement` (如 `readout.temperature(...)`) 默认 `display=True` → 自动出适配的默认图。但在 task console / GUI 里, `measurement` 作为 logic 节点由 `ScannedMeasurementNode.step()` 逐点驱动、**只 publish 到 hub、从不调 .run()** ⇒ 等价 `plot=False`: 不自动出图, 用户自己加 Plot 面板按 signal 配。

5.3 Processor: 逐帧/反应式变换 (“func” 层)

Processor (`LogicNode` 子类) **无设备采集**: 消费一个或多个 hub 信号、计算、republish 派生信号。它是**反应式**的——只在某个被消费信号**自上次以来前进了**才发 (按 hub 每信号版本判断), 所以它作为一个 live 图节点跑在产它输入的 `measurement` 旁边, 自有轮询节律, 没新数据就 no-op。

OccupancyProcessor 是**真读出流程**的逐帧节点: 消费相机 `frame`、跑与真机读出**相同的** `calibration.detect` 契约 (不重实现任何检测数学), 每新帧发布 `occupied(N)` / `counts(N)` / `rate` (标量 EMA 装载率) / `rate\_sites(N)` / `rate\_grid` (网格) / `centers(N,2)` / `thresholds(N)`。**这正是 virtual==real 的拆分**: 相机 `measurement` 只发 `frame`, `detect` 是**单独**的节点; 标定 (站点中心 + 逐站阈值) 来自先跑的 `calibrate-readout` task, 与真机完全一致。

5.4 Task: 一次性编排

Task (`LogicNode` 子类) **编排**多步流程 (采帧 / 标定 / 文件夹读取), 可向一张专用面板发**中途输出** (confocal 式)。one-shot: `step()` 跑完整个 `run(out)` 一次, 自停。**task 向 hub publish 0 个信号**——结果/标定留在实例 (`node.result/node.calibration`), 中途帧/进度写进它自己的 `TaskOutput` 缓冲 (`node.output`, 非 hub)。控制台把 `node.output` 绑到一张固定面板 (保留键 `\_\_task\_frame\_\_`) 显示它跑时的过程。

5.5 装载读出 = 三个节点自己拼

看 loading**没有一键复合体** (没有 `build\_loading\_readout`、没有自造的“Readout: Loading”复合节点): 你用三类节点**自己拼**——

1. **Measurement: Camera** (live frames) (`CameraMeasurement`, 发 `frame`);
2. **Processor: Judge occupancy** (`OccupancyProcessor`, 逐帧跑真 `calibration.detect`);
3. **Task: Calibrate readout** (`CalibrateReadoutTask`, 先跑、写回标定 / 产 npz)。

各自 Start, 再在 Monitor 加 Plot 面板按 signal 连。**每层独立、信号显式相连, 没有单体节点一次造出所**

有信号——每个节点只经 `CameraDevice` 契约取帧，换真机只换传入的 `camera`（唯一虚拟之处 = 相机数据源）。这一点很关键：节点不是“假数据”通道，它走的是和真机一样的检测原语，所以面板表达式、拟合、站点图在虚拟上验证通过 = 在真机上同样成立。

5.6 Judge occupancy 反应式处理器

`Judge occupancy (operations/processors/occupancy.py)` 是目录里的主读出 processor，build 一个 `OccupancyProcessor`。声明参数：

calibration (Calibration file) 一个已存标定的路径（`.npz/.json`：站点中心 + 逐站阈值 [+ PSF 权重]，如 `calibrate-readout task` 的产物）；**留空 = 用当前会话标定**（懒取：`calibrate task` 还在跑时 detector 先 no-op，标定就绪后自动接上）。

source (Frame signal) 携带相机帧的 hub 信号名（默认 `frame`）。

ema (Rate smoothing) 滚动装载率的指数滑动平均权重。

它消费每个 `frame`、经真 `calibration.detect` republish 上面那组逐站信号；默认视图是按占据着色的 `sites` 原子图。（目录里还有一个一次性的 `Readout fidelity` processor，在一个**已存文件夹**上跑逐站读出保真度刻画——见下面“扩展到真实设备”。）

5.7 多节点做 A—B

给同一台实验的两路读出分别用无前缀与 `b\_` 前缀的节点，于是 `value = rate\_grid - b\_rate\_grid` 这种跨实验相减开箱即用。

6

读出标定 task 与脉冲编程接口

6.1 calibrate-readout task 的完整参数设计

`CalibrateReadoutTask` (`operations/logic.py`) 跑真的 sitemap + 逐站阈值标定——就是 notebook 的 `exp.readout.sitemap()/thresholds()` 流程——产出一个 `TrapCalibration` 供下游 `OccupancyProcessor` 用。所有可调参数在 `operations/tasks/calibrate.py` 的 `CALIBRATE\_PARAMS` 声明一次 (GUI 自动表单与 `readout.calibrate\_task(**values)` 同读):

source = live / folder live = 现在开相机直接采图; folder (+ `data\_dir`) = 用已存文件夹的图 (`index\_run` 读 `img<n>`) 标定。

两条 pulse (两个 readout duration) **sitemap 这一遍用较长 readout** (更多光子 → site 中心 + PSF 拟合更干净) , **threshold 这一遍用实际 readout** (阈值要在真实读出条件下学) 。各自: `sitemap\_exposure/readout\_exposure` (默认序列时长) 或 `sitemap\_pulse/readout\_pulse` (已存 `PulseTableState` 的 .json, 经 `PulseTableState.load().to\_seq` 当成像脉冲, 空 = 默认序列)。 `\_imaging\_seq(pulse, exposure, name)` 单点选择。

mode = box / per-site PSF / uniform PSF 经 `MODE\_TO\_SITEMAP\_METHOD` 解析为 `sitemap` method `box/psf/uniform\_psf`。

threshold_method = otsu / bimodal otsu = 单点分割; bimodal = 逐站暗/亮高斯核拟合。

save_path / load_path 存 / 取标定 (`TrapCalibration` = 站点中心 + 逐站阈值, 经 `TrapCalibration.save/load`); 给 `load\_path` 直接复用已存 site+threshold、**不重新采图**。

calibration_frames / threshold_frames / roi_radius / grid_shape 模板帧数 / 学阈值帧数 / 逐站方 ROI 半宽 / 阵列形状。

同一 `TrapCalibration` 契约(`calibrate\_sitemap\_from\_images / calibrate\_threshold\_from\_images`), 虚拟与真机只差相机帧。中途经 `TaskOutput` 把模板帧 + 进度发到专用面板 (`mid\_run = ("frame", "progress")`, `cal\_命名空间`), **不进 hub**。

6.2 脉冲编程接口对齐 GUI

`PulseTableState` (`neutral\_atom/timing/pulse\_table.py`) 可被**程序化驱动**成与脉冲 GUI 完全一致——于是是一个 `measurement / task` 能在代码里 `load` 一个脉冲、改几个字段再当成像序列用（如上面 `calibrate task` 的两条 `pulse`）：

`load(path) / save(path)` 与 GUI 同一 `.json`：读回 / 存出。

`set\_period\_duration(i, dur, unit=)` / `set\_period\_name(i, name)` 改某周期的时长 / 名字。

`add\_period(dur, name=, unit=, ...)` 追加一个周期。

`set\_period\_state(i, channel, value)` 设某周期某通道电平。

`set\_channel\_delay(channel, delay, unit=)` 设通道延时。

`bind\_field(kind, target, ...)` 把某字段绑成可扫量（`duration/dac slot`），与扫描读出时长同一种可扫量。

`set\_scan\_table(rows)` 设流式扫描表。

`to\_sequence(...)` 编译成 `PulseSequence` 下发或当成像脉冲。

所以“在 GUI 里能拼的脉冲，代码里同样能拼”——`measurement / task` 在代码里 `load + retune` 一个脉冲，与 GUI 走同一模型、同一 `to\_sequence`，不会漂移。

7

面板模型

7.1 六种面板类型 (kind)

kind	绘图类	用途
2d	Live2DDis	图像 + 侧分布 + colorbar + 可拖 clim (相机帧、网格率)
sites	LiveSiteMap	原子阵列站点图: 每站一个实心圆按值着色 + 可垫相机帧
1d	Live1D	一维折线 (逐站量、横轴站号)
monitor	LiveLiveDis	标量滚动轨迹 + 侧分布
monitor_nodist	LiveLive	滚动轨迹无侧分布 (独立 plot type; LiveLiveDis 继承它加分布带)。装载率等侧分布无物理意义的量用它
hist	HistogramFigure	直方图 + 自动双高斯 + 可拖阈值线 + 保真度

7.2 来源 = 表达式

每个面板的数据来源是一行 Python, 赋给 `value` (与脉冲 GUI 的 Scan 页同一套“本地可信代码”信任模型——只跑你自己写的):

Python

```

value = frame                # 最新相机帧 -> 2D
value = occupied             # 每站 0/1 占据 -> Site map
value = rate_sites           # 逐站装载率 -> Site map / 1D
value = history('counts', 200).ravel() # 最近 200 shot 计数 -> Distribution
value = rate_grid - b_rate_grid      # 两个实验相减 -> 2D

```

命名空间 = hub 的一致快照 (每个信号名直接可用) + `history/latest/names/shot/np/math`。

7.3 错误隔离与形状自适应

`PanelCard.refresh(namespace)` 把表达式求值与喂图包在 try 里：某个面板表达式报错，错误只显示在该面板自己的状态行（红字），其余面板照常刷新，控制台循环不受影响。`\_coerce` 按 kind 校验/降采样 value（2D 对 >192 维做步进降采样保证每 tick 便宜；sites 限站点数）。`\_render` 里只有当 value 形状变化（换表达式/尺寸/参数）才重建该面板的图，否则增量更新。

8

面板几何:confocal 模数规则

这是面板能整齐拼接的根本，也是 frontend 密封 API 的核心体现。

8.1 所有 kind 共享同一块 plot 区域

`panel\plot\_spec(size)` (`frontend/live.py`) 对**每一种 kind** 返回同一套几何：

- `PANEL\_SIZES = ("1x2", "2x2", "1x4", "2x4")`，含义是“行 × 列的半单元数”。
- `2x2` = frontend 的标准 plot 区域，数据区 480×360 ，边距 `PANEL\_MARGINS\_PX = (110, 86, 80, 70)` (左右下上)：左 110 取自 `STOCK\_MARGINS\_PX` 的 confocal 左边距，是容纳 4–5 位 y 刻度标签 (如 qCMOS 像素 1180) + 旋转 y 轴标题**不被裁切**的最小值；右 86、下 80 相对 confocal 收紧以提高数据占比；顶 70 = `TITLE\_SLOT\_PX` 的 title 槽，常备，与有没有标题无关。
- `1x2` = 半高，`2x4` = 双宽。
- 2D 与 Site map 在**自己的区域内部**按 `\_IMAGE\_SPLIT` (`[0.75, 0.1, 0.1]` = 主图 + 侧分布 + colorbar) 水平分割，所以 `2x2` 主图恰好 360×360 **正方形**且与 colorbar 顶底对齐——这正是不同类型面板能整齐排版的原因。

对外接口只有 `panel\plot(data, kind=, size=)` 与 `panel\plot\_spec(size)`：**尺寸是唯一旋钮**，几何/dpi/边距不可外部配置 (`panel\_size\_cells` 校验尺寸，非预设抛错)。

8.2 显示缩放:qt_canvas 三不变量

面板图通过 `qt\_canvas.panel\_canvas(figure)` 嵌入，封装常数 `PANEL\_DISPLAY\_SCALE=0.7`。`EmbeddedFigureCanvas` 维持三条不变量 (在 `QT_SCALE_FACTOR` 1.0/1.25/1.5 三档离屏复现 + 验证)：□ `figure.inches` 构造后永不变；□ `fig.dpi` = 设计 dpi × 真实屏幕比例 (标准 retina，显示因子绝不进 dpi)；□ 控件逻辑尺寸 = 设计 px × `display_scale`。`resizeEvent` 故意不从控件尺寸反推 figure——这是高 DPI 屏幕下布局不变形的关键 (见性能/契约章)。

9

卡片与布局

9.1 卡片 = 带标题的 Fluent 卡

`PanelCard` 继承 `FluentGroupBox`, 卡框标题 = 面板类型 (2D image / Site map / ...), 带 fluent 阴影 (`CachedDropShadow`, 按 silhouette 缓存、move 时不重新模糊)。画布下方、卡片底部一行灰色状态行 (当前 shot + 来源表达式回显)。标题行右端一个 `Setting` 文字按钮 (出错变红、错误进 tooltip)。

9.2 模数闭合:footer 吸收差额

图本身 = plot 区域 + 边距, 边距不随数据区折半, 所以**光靠 canvas 永远对不齐网格**。卡框的状态行区按行数**吸收这点差额**: `\_card\_size(size) = rows$\times$ $pitch\_y - gap`, 使得**一张 2x2 卡严格等于两张 1x2 卡叠起来加一道间隙** (竖直 pitch 323、水平 259, 契约测试锁定 $2 \times \$315 + 8 = 638$)。任意尺寸混排严丝合缝。

9.3 拖动吸附 + 重力压实

排版不用填行列数字: **按住卡片边框 (画布以外的空白) 拖动**, 松手 `\_pos\_to\_slot` 吸附到网格槽并写回 config。布局走**重力压实** (`\_compact`) —— 每张卡按读序 (row→col) 在**自己的列内向上落到上方卡或顶端为止**。这一**规则同时**做了两件事: 卡总是落在与它共列的卡**严格下方** (无重叠), 且**不悬空在空位之上** (无竖向空隙), 所以结果恒为顶部紧凑、无洞的网格。刚拖动的 `active` 卡**平局优先**——保住放下的列/行, 其余卡绕它回流。删卡或拖走后留下的空位被自动回填, 整板始终顶部紧凑、无洞。

10

Setting 弹窗: 瘦身后的基本配置

Setting 弹窗按 `confocal_gui` 的**竖排范式**排布——粗体小标题占独立一行 `FluentSectionLabel`, 下方每行 `[64px 标签 | 控件]` (`FluentSettingRow`)。轻量、高频的“换数据 / 换显示 / 切单位 / 钉 lim”留在 Setting 即时生效; 重量级处理 (命令行拟合 / `reim` / Measurement) 在**每面板的 Edit 标签** (见第 11 章), 避免弹窗过载。

卡片标题行的 **Setting** 按钮打开该面板的设置弹窗 (`confocal` 风格), 从上到下五段——**Source / Display / Unit / Limits / Panel**, **全部即时生效**:

Source `signal` 下拉列出 `hub` 此刻真实存在的所有信号名——选一个面板立即切到 `value = 信号名`; 要做运算就在表达式框写一行点 **Apply**。出错时 **Setting** 按钮变红。

Display 一律**即时生效**:

- `size` 尺寸预设。
- `colormap` (仅 2D Site map) ——这就是**改 colorbar 配色 (colorset)** 的地方; **没有“显隐 colorbar”开关** (带 colorbar 的图恒显 colorbar)。
- `colormap` 之外的功能参数 (Site map 的 `centers/image`、Rolling 的历史长度、Distribution 的 bins) **不在此处**——它们是 `display=False` 的 `PANEL\_PARAMS`, 渲染在该面板的 **Edit 标签** 的 Parameters 段 (避免与 Setting 重复)。

reim `lim\_combo` 给两档 `tight/normal` (`confocal_gui combo\_reim` 同款命名), 写 `config.params["reim"]` (重建面板时重应用)。两档语义集中在 `BaseLivePlot.\_mode\_target`**单一真相源**: `normal` 把下界钉在 0 (数据非负时, 读数对着零比), `tight` 上下各留 10% 余量裹住数据。**对带 colorbar 的图 (2d / Site map), 色限 clim 就是 y 轴的二维对应物**, 同样走 `\_mode\_target`: `normal` 把 colorbar 锚在 0、`tight` 裹住帧值; 切换时 `Live2DDis.apply\_reim\_now` 即时重映射 colorbar (两个方向都重映), 并同步可拖 `clim` 线。

Unit **Unit** 按钮在 `DataFigure.change\_unit` 的单位环上前进一档 (GHz/nm/MHz 或 ns/us/ms 等); 右侧显示当前累计倍数 (`unit\_index`)。轴标签未带可换单位时是 no-op。

Panel (这张卡) 面板标题(显示在图内, 必须经 `BaseLivePlot.\_apply\_title` → `style.apply\_title` 改才有正确字号)、**Remove**、**Edit...** (开**该面板专属的 Edit 标签**做参数 / 全套拟合 / 手动 lim /

Source
accepts a per-site (N,) occupancy vector
(centres + frame come with it)

signal occupied — occupancy ...

value = occupied Apply

shot 81

Display

size 2x2

colormap gray

relim tight

unit Unit unit

Panel

title Per-site occupancy

Remove Edit... Save Fig

Figure 10.1: confocal 式竖排 Setting 弹窗: 粗体 `FluentSectionLabel` + 每行 `FluentSettingRow` [64px 标签 | 控件]。五段从上到下:Source(signal 下拉、表达式 + Apply、status)、Display(size + 各 kind 声明式参数含 colormap 配色)、Unit(`Unit` 按钮 + 当前倍数)、Limits(mode + 3 列小 grid: 轴 / lo / hi)、Panel(标题 + Remove | Edit... | Save Fig)。无 colorbar 显隐勾选——colormap 即是 colorbar 配色选择器。圆角卡由 `FluentPopup` 的 `paintEvent` 画, 无边框样式表可级联到子控件。

详细处理)、Save Fig (一键导出 png + npz 到 tasks/)。

Note

坑 (已修): 弹窗用 `FluentPopup (qt\_fluent.py)` ——**半透明 + 无边框**的顶层 `Qt.Popup`, 圆角白卡由 `paintEvent` 抗锯齿描一笔。两条教训: □ 不可在**不透明**顶层弹窗上用样式表 `border-radius`——方角白块会露在圆弧外 (右下角那条“奇怪的线”) + Windows 原生阴影; □ 光加 `WA\_TranslucentBackground` 又会让样式表背景**整块不画** (弹窗变透明)。所以圆角卡自己画; 边框是画出来的、不是样式表规则, 因此也绝不会级联到子控件。

10.1 持久化:Setting 的每一项都进 PanelConfig.params

Setting 改的每一项——`size`、`colormap`、声明式参数 (`bins` / 历史长度 / `centers` / `image` / `cmap`)、**re-lim 模式 (与 Edit 标签同 key)**、Unit 的 `unit\_index`——都写进 `PanelConfig.params` (连同标题、源、网格槽), 随布局 JSON 一起保存。面板因尺寸 / 表达式变化而**重建**时, `\_apply\_display\_params` 在 `\_build\_plot` 收尾时调一次, 把 `reim\_mode` 与 `unit\_index` 重新应用到新建的 plotter 上 (`cmap` / `bins` 等声明式参数则直接进 `panel\_plot(...)` 的 `kwargs`)。所以“我把这张卡换了配色、调了 `bins`、切到 `normal + us` 单位”在保存/载入/重建后都保持。每面板 Edit 标签的 Processing 段做的是另一份**冻结快照** (照搬该卡 `size`) 上的细看, 包括手动钉 `x/y lim` 的功能; 它不写回 Monitor live 卡的 `config`, 二者不冲突。

10.2 声明式参数: 加一个旋钮 = 一行

`ParamSpec(key, label, kind, default, choices=, lo=, hi=, tooltip=)`, `kind` 取 `choice/int/signal`。Setting 控件、JSON 持久化、改动后重建全部自动跟上; `\_set\_param` 是唯一写入口。要给某类面板加一个可配置项, 就在 `PANEL\_PARAMS[kind]` 加一行——不用写 bespoke 控件。

11

两个常驻标签 + 每面板/每节点 Edit 架构

控制台是一个 `FluentTabWidget`，有两个常驻（不可关）标签 + 按需的每面板/每节点 Edit 标签：

Monitor（常驻，不可关） = plot 面板的拖拽 live 网格，纯做实时监控。面板、Setting（基本配置控件）、拖动吸附 + 重力压实都在这里。

Logic（常驻，不可关） = measurement / processor / task 逻辑节点的列表，每行一个状态点（`LogicNodeRow`） + Edit。控制台开局空 hub、不自动启动任何东西。

从 **Add Panel** 建节点 连了会话后，标题栏的 **Add Panel** 下拉把五层架构里每个可加的节点按层列出，而不只是 6 种 plot kind: `Plot: <kind>` (纯视图) / `Measurement: Camera (live frames)` + `Measurement: < 扫描名 >` / `Processor: < 名 >` (如 `Judge occupancy`) / `Task: < 名 >` (如 `Calibrate readout`)。measurement / processor / task 三份目录都是自动发现的(来自 `measurement\_specs()` / `processor\_specs()` / `task\_specs()`)。选一个 **Add Panel**: plot 在 Monitor 板加一张视图，logic 节点在 Logic tab 加一个默认停的行并打开它的 Edit。没有单独的全局发起器，没有自造“Readout: Loading”复合体。

每面板 **Edit** 标签 (**PanelEditor**，可关闭) = 点面板的 **Edit...** 多开一个可关闭标签 (右上一个柔和的自定义 × 关闭键，灰色悬停才高亮，非刺眼原生红叉)，承载那个面板专属的：它所读信号的产出节点自报的源采集参数 (Acquisition + Apply) + plot 的 API 参数 (自动发现) + 全套 data_figure 拟合 + 手动 x/y lim + Save Fig; 无 Start/Stop (节点在它自己的 Logic Edit 起停)。

每节点 **Edit** 标签 (**LogicNodeEditor**，可关闭) = 点 Logic 行的 **Edit...**: 该节点自己的参数自动表单 (measurement/processor/task/camera, 相机来自 `readout.camera\_spec()`) + **Start/Stop** (task 的 Start 即 Run); 无 fit、无 limits (拟合是 plotter 的事，去 Plot 面板做)。

理由: 严格落实“每个面板/节点各自 edit, 方便实验”的设计意图; 可关闭标签复用 `FluentTabWidget.add\_closable\_tab` (frontend 件, 自定义柔和关闭键; 裸 `addTab` 无关闭键, 故脉冲 GUI 的 Edit/Preview/Scan 三页永不可关); Edit 内容不与 Setting 重叠 (源/尺寸/colormap/relim/unit 只在 Setting, 功能参数只在 Edit)。

11.1 VIEW / LOGIC 解耦

加一个 Plot 面板 = 纯 VIEW, 不会启动任何东西: 它一开始空白, 只有在 Setting 里配了 signal 且产出该 signal 的逻辑节点被 Start 之后才显示。空 hub 时 plot 的 signal 选择器只有 (expression), 不会凭空冒一堆 signal。加一个 measurement / processor / task = 一个 Logic 节点行进 Logic tab, 默认停 (行点灰 GREY); 在它自己的 Edit 里 Start (行点绿 GREEN) / Stop; 出错红 RED (LogicNodeRow.STATE\COLORS)。控制台两套集合: `console.logic\_nodes` = Logic 行 (声明, 不管跑不跑都在); `console.running\_nodes` = 当前在跑的已建节点 (Start 时 append、Stop 时 remove)。



Figure 11.1: 一个 plot 面板的 Edit 标签 (可关闭, 标签右上一个柔和的 × 关闭键): 顶部 Acquisition 段是它所读信号的**产出节点**自报的源参数 (相机 = exposure/region) + Apply (原地下发); Parameters 段是该 plot 的功能参数 (自动发现); Fit 段是**全套 DataFigure 模型** (含 2D center, 不按 kind 门控) + 自由文本 args + Fit/Clear, 再加手动 x/y lim; Processing 段是冻结快照 + Refresh + Save Fig。整页可滚动。**plot 面板绝不内嵌某测量的“参数表单 + Start”**——那在该 logic 节点自己 (Logic tab) 的 Edit 里。

11.2 task 运行接管控制台 (confocal 式)

Start 一个 task 时, console 在 Monitor 开一张**固定**面板显示它的中途输出 (从 `node.output` 读, 保留键 `\_\_task\_frame\_\_`, **不经 hub**): `TaskSpec.mid\_run\_key` 选缓冲键 (如 `frame`), `\_refresh\_task\_panel` 在 `\_tick` 的 version-gate 之前刷 (因为 task 输出不 bump hub version)。同时顶部升起橙色横幅 “Task running ...%”, 并**禁用其他一切操作** (Add/Save/Load/Edit/其他节点 Start), 只留 Stop (`console.\_task\_locked` 把所有入口 short-circuit 掉)。task 完成或 Stop 后**解锁**并撤掉该瞬态面板。这样一次性标定/采集流程跑时, 操作者只盯它的进度、不会误触别的节点; 它的瞬态帧也永不混进 live 读出信号。

11.3 每面板 Edit: 冻结快照模型 (关键设计)

点面板 `Edit...` 打开它的 `Edit` 标签时, `PanelEditor` 绑定该面板当前数据的一份快照——用 `panel\_plot` 照搬 `Monitor` 那张卡的 `size` (`size=card.config.size`, 与活面板同尺寸) 重建一个独立的 `plotter` (不是 `Monitor` 卡那个), 整页放进 `FluentScrollArea`, 所以快照再大也不会把底部控件挤出可视区, 且快照与活面板尺寸一致。选择器只在 `Edit` 启用: `Edit` 快照 `interactions=True` (缩放 / 拖阈值线 / `AreaSelector` 框选), `Monitor` 卡 `interactions=False` + `isolate\_wheel=False` (滚轮滚仪表盘、不被框选吞)。理由:

- 天然契合“冻结细看” workflow: 看的是一帧定格快照, 不会被实时刷新搅动 (`Refresh` 才重抓)。
- 不打扰仪表盘: 拟合/改范围都作用在 `Edit` 自己的图上, `Monitor` 照常实时刷新。
- 每面板各一份: 编辑器按 `id(card)` 注册 (`\_panel\_editors`), 重点 `Edit` 聚焦同一标签不重复; 关闭 (X) 或删除面板即 `teardown`。

11.4 Edit 控件 (详细参考 confocal)

11.4.1 plot 面板 Edit (`PanelEditor`)

Acquisition (编辑数据源的采集参数) 一个 `plot` 面板是视图; 它读的信号由某个逻辑节点生产, 背后是数据源——相机本身, 或一个变换节点。Edit 顶部列出数据源的可编辑参数: `\_producing\_node(card)` 用 `node.published\_signals()` $\cap$ 面板表达式引用的信号名找到产这张图的逻辑节点, 节点经 `acquisition\_parameters()` 自报哪些可调 (由源决定, 不靠反射 `\_init\_`)。一张原始相机帧面板的源是 `CameraMeasurement`, 列出 `exposure` / `region` (ROI 端点) / `frames\_per\_cycle`; 一张占据面板的源是 `OccupancyProcessor`, 列出它的 `detect` 设置。每格预填当前值并在下标 `now: < 值 >` 供改参参照; 点 `Apply` 经 `set\_acquisition\_parameters(...)` 把改动原地推给源——相机就 `camera.configure(...)` 实时重配——同一节点以同一信号名继续发布, 不重启节点 (节点从它自己在 `Logic` tab 的 `Edit` Start)。plot 面板绝不自带某 `measurement` 的“参数表单 + Start” (`PanelEditor.meas\_panel` 恒 `None`); 要重跑一个扫描就去 `Logic` tab 调那节点。

Parameters (plot 的 API 参数) 该面板 `plot` 调用的可变参数自动铺成 GUI 控件 (`length/bins/centers/image`, 从该 `kind` 的声明 `PANEL\_PARAMS` 自动发现, 复用 `\_make\_param\_widget`); 改一下经 `card.\_set\_param` 实时重渲染活面板并重抓 `Edit` 快照。 `cmap` (`display=True`) 是唯一留在 `Setting` 的; 其余 `display=False` 的功能参数都在这里——两边永不重复。

Fit (全套 data_figure, 不门控) 模型下拉是完整的数据图集合: `Lorentzian` / `Gaussian` / `Lorentzian (Zeeman)` / `Rabi` / `Exp decay` / **2D center, 对所有 plot 类型都可用** (`DataFigure` 从快照自取 1D/2D 路径——2D 图用 2D 高斯 `center` 拟合, 其余拟合 1D 轨迹)。一行自由文本参数注入拟合调用 (`eval(f"\_df.\{method\}(\{cmd\})")`), 与 `confocal` 的: 框同理): `p0=[...]` 初值、名字 = 值固定, `is\_fit=False` 只叠加。 `Clear` 移除; 底层 `DataFigure`。

手动 x/y lim 四框 + `Apply lim` (`DataFigure.xlim/ylim`); 打开时自动回填当前坐标范围。 `relim/unit` 不在 `Edit`——它们在 `Setting`, 避免重复。

Save Fig < 标题 >.png + 数据 .npz (时间戳) 到 `tasks/` (`DataFigure.save` 返回 {figure, data})。

11.4.2 logic 节点 Edit (LogicNodeEditor)

Logic tab 上 measurement / processor / task / camera 节点的 Edit: 该节点 `ParamDecl` 自动表单 + **Start/Stop** (task 的 Start 即 Run), **无 fit / 无 limits**——拟合是 plotter 的事, 去 Plot 面板做。两类 Edit 因此各属一套类 (`PanelEditor` vs `LogicNodeEditor`), `do\_fit/fill\_limits` 等只在 plot 面板 Edit 上有; fit 由**面板角色**门、绝不由 plot kind 门。

Note

解耦要点: `PanelEditor` 持有**自己的**独立 plotter / `DataFigure`, 只读 `card.plotter` 取数据, 与 Monitor 卡解耦——它不通过 `card.\_set\_param` 去改 Monitor 卡 (那条路径会 `\_reset\_plot` 把 Monitor 卡的 plotter 置 None, Edit 的快照随即失去依据)。逻辑节点的生命周期完全归 `LogicNodeEditor` (Logic tab): 关一个 plot 面板的 Edit 不影响任何在跑的节点。

11.5 坐标轴 = 源的参数空间; 缩放/框选回写 + 采集环应用 (confocal 核心耦合)

Confocal_gui 的核心思想: 一张图的坐标轴**就是**产生它的源的参数空间;**改变可视范围**就把“该轴代表的源参数”回写成“下次采集只覆盖这块”, 而**重新采集**才让活面板真正换上新参数的数据。两件事缺一不可, 控制台对每种空间/参数轴都落实同一条耦合。

(1) **缩放或框选 → 回写参数 (view-limits-else-area 优先级)**。confocal 把**同一个回写回调**同时绑到**滚轮缩放/中键平移 (ZoomPan)**和**左键框选 (AreaSelector)**两个交互上。**选择器是一个通用接口**: 它**永远只产出矩形的四个端点**(`x\_min, x\_max, y\_min, y\_max`) (或 1D 的两端点), 坐标是图的轴单位, **不懂相机/ROI/扫描格**——它服务每一种 2D 面板 (相机帧、二维参数扫描……)。取值优先级:**画了框就用框选矩形, 否则用当前可视范围**(`ax.get\_xlim()/get\_ylim()`)。所以光滚轮缩放就能更新范围, 框一个矩形则覆盖视野钉得更精确。**控制台只把这个矩形交给“源”, 由源把它转成自己的参数格式**——前端绝不内嵌任何设备特定形状:

1D 扫描曲线 → 扫描范围 扫描结果的横轴是**真实物理量** (`t_off` 秒、`detection time` 秒, 见 (N, 2) x-y 节), 不是 0,1,2 index。Edit 图上**滚轮缩放或框一段**, `\_read\_scan\_range` 把端点写进**这条曲线的扫描节点** (Logic tab) 的扫描 Min/Max 表单——宽扫一遍 → 缩放/框住特征 → 在它自己的 Logic Edit Start 只重扫这一段。

2D 面板 → 源自报的参数 (全程 plot 坐标) `\_read\_region` 把矩形端点交给产这张图的源节点 `node.region\_to\_acquisition\_parameters(x\_min, x\_max, y\_min, y\_max)`; **采集层的参数本身就是 plot 坐标**——相机帧面板 (源 `CameraMeasurement`) 的空间参数是 `region = [x\_min, x\_max, y\_min, y\_max]`**端点**(不是设备的 `[x, w, y, h]`), `acquisition\_parameters()` 报它、`set\_acquisition\_parameters(region=...)` 收它, Edit 框里显示的就是端点 (与选择器一致)。**端点 → 设备 ROI 的转换只藏在内部**(`set\_acquisition\_parameters: region → [x, w, y, h] → camera.configure(roi=...)`), 再由**相机设备层**吸附到**子阵网格**(qCMOS SUBARRAY* 须 4 的倍数, 见设备层节) 并**回读**实际窗口。**别的源各转各的**: 二维参数扫描返回两条轴范围而非 `region`——同一选择器/同一条 `\_read\_region` 服务所有 measurement/acquisition, 不被任一设备格式写死。相机帧图像 x/y **就是真实像素坐标**: `\_coord\_frames` 取源 `region` 端点 (`index 0=x\_min, 2=y\_min` 即原点) 建网格, 轴标 `Camera x/y (px)`; 像素坐标常 4-5 位, 2D 图复用智能刻度**对**(`apply\_smart\_ticks`) 收紧刻度, 宽标签不挤糊。`Refresh(rebuild)` 先 `refresh\_once` 推一拍, 快照取的是**最新一帧**而非上一节拍的渲染。

(2) Apply / Start → 把改动应用到采集, 活面板才换上新数据。回写只改了参数框的值; 真正让 Monitor 显示新参数下的数据, 要把改动应用到正在采集的数据源。confocal 的 `on\_start` 对一次有限扫描是用新参数重建测量再跑; 对一台持续流式的相机, 则是让采集环(数据源的唯一持有者) 接住改动、在两帧之间 re-arm。两者都**绝不是**“另一个线程原地改一下还在跑的那台相机”。控制台据此分两路:

- 扫描 measurement: 在它**自己**的 Logic Edit 里用表单当前值 Start 一个 `ScannedMeasurementNode(spec.build(**val` 得未跑测量再包成节点) 重跑——plot 面板 Edit 不内嵌它的表单。
- 相机/采集源: plot 面板 Edit 的 Acquisition 段 Apply 经 `node.apply\_acquisition\_parameters(...)` 把改动交给采集环。采集环在运行时是数据源的唯一持有者(唯一调用 `acquire/configure` 的线程): 改动被排队, 由采集环在两帧之间用它自己的线程应用 (`set\_acquisition\_parameters`)——流式相机于是在自己的线程里干净 re-arm, Monitor 持续刷新、下一帧换上新视场。**绝不从 GUI 线程停/启那条采集线程**: 那会 (a) 在 `join` 上卡住 GUI, (b) 若阻塞的 `acquire` 没及时响应停止, 就会有第二个 `acquire` 与旧线程并发跑在同一台相机上 → 死锁/冻结 (这正是“Apply 后 Monitor 暂停”的根因)。节点空闲时 `apply\_acquisition\_parameters` 直接原地 `set\_acquisition\_parameters+step()` 一帧并推进 epoch。每个采集参数右侧的 `now: <值>` 引用由 `TaskConsole.\_tick` 每拍刷新当前可见的 Edit 标签, 所以排队的改动一旦被采集环应用 (`acquisition\_epoch` 推进), `now`: 自己就跟上——这也避免硬连线碰坏 Edit 既有的 Refresh/Fit/Save 控件。

此外 `show\_task\_console` 在开窗时会把 `running\_nodes=` 里传入但未运行的逻辑节点逐个 `start` (幂等, 已自起的保留各自速率), 所以“把节点交给控制台 ⇒ 开窗即 live”; `TaskConsole.\_\_init\_\_` 故意不这样做, 以保 notebook/测试的确定性手动 `step()`。

Note

为什么是统一的架构而非各打补丁: 坐标系信息走一条与 1D 扫描回写同形的解耦缝——`TaskConsole.\_coord\_frames()` 把“每个声明了 ROI 的节点的 published 信号 → 该 ROI”汇成保留键 `\_\_coord\_frames\_\_` 注入表达式命名空间; `PanelCard.\_source\_coord\_frame` 用面板源表达式引用的信号名 (`\_co\_names`) 在里面查, 任何空间轴的面板都能问“我的源坐标系是什么”。`frame` 形状不变但 ROI 平移时, 面板按坐标系变化 (不只看形状) 触发重建, 轴永远跟着 ROI 走。回写 (缩放/框选, view-else-area) 与应用 (扫描节点 Start 重建测量 / 相机 Apply 交采集环在两帧间 re-arm) 是两件正交的事, 前者改参数、后者才换数据——两者都缺会让“改了却看不到效果”。采集环 = 数据源唯一持有者的不变量 (改动只由它本线程在两帧间应用, 外部一律排队) 是统一保证: 任何节点改参数都不会冻 GUI、不会两个 `acquire` 撞车, 不只针对 qCMOS。这条缝对未来其它“轴 = 源参数”的图同样可用, 不必每种图各写一遍。

12

通用 Measurement 框架

通用 Measurement 框架把“**扫一个被绑定的脉冲参数、每点采几帧、把帧约简成一个数（或逐站点向量）做成 live 曲线**”抽象成**一套引擎**，让读出时长 → 保真度、release-recapture 测温这类测量都是**同一形状**，而不必各写一份扫描循环。代码在 `operations/measurement.py` (引擎 + 声明)、`operations/logic.py` (`ScannedMeasurementNode` 节点包装)、`subsystems/readout.py` (单一真相源建器 + 目录)。

12.1 三角色解耦: Axis × Plan × Reducer

一个被扫描的测量，差别只在三件事，各抽成一个角色：

ScanAxis(slot, values, label, unit, kind) 扫什么: 绑哪个 named-slot (`duration` 或 `dac`)、扫哪些值。时间轴用 `pulse.set\_time` (用户单位经 `scale\_to\_ns` 换 ns), DAC 轴用 `pulse.set\_slot` (带符号用户值)。

ShotPlan (协议) 每点采几帧、序列怎么生成: `n\_frames + sequence\_for(pulse, axis, value)` (只用 `frame\_sequence` 契约)。NFramePlan 是“N 独立帧、每帧一触发” (读出时长扫描用); ReleaseRecapturePlan 是“一次 acquire 取回两帧” (测温用, 见下章)。

PointReducer (协议) 帧怎么变成 y: `reduce(frames, calibration)` → 标量或逐站点向量。`OtsuFidelityReducer` 把帧的 `calibration.signals` 池化、Otsu 分割、出双高斯保真度; `SurvivalReducer` 出存活率。

握住这三者、共享一台引擎，意味着**加一个新测量 = 写一个新 reducer/plan, 而不是又抄一遍 live-scan 循环**。

12.2 引擎: ScannedMeasurement / ScanResult

`ScannedMeasurement(pulse, camera, sequencer, calibration, axis, plan, reducer, shots\_per\_point)` 是引擎：

- `measure(value, index): axis.apply` (设 slot) → `plan.sequence\_for` → `camera.acquire(plan.n\_frames, sequence=...)` → `reducer.reduce(shots\_per\_point` 次取均值)。这就是 **live 引擎每点调的那个回调**。

- `run(live=...)`: 注册了 viewer 时交 `active\_plotter().run` (后台 worker、UI 线程刷新、协作 `stop()`); headless 时同步跑同一回调, 仍返回完整 `ScanResult`。

`ScanResult`(继承 `MeasurementTaskResult`)携带 `x + data\_y((n\_points, n\_series):` 标量 reducer 填第 0 列, 逐站点 reducer 每站一列), 白拿 `stop()/points\_done/running`。

12.3 声明式单一真相源:MeasurementSpec / ParamDecl

为了让 **API 默认值与 GUI 控件不漂移**, 一个测量把它的参数**显式声明一次** (不是签名反射/AST——避开 confocal 的 AST 猜 caller 坑):

ParamDecl(key, label, kind, default, unit, lo, hi, required, choices, tooltip)

一个可扫/可调参数的声明。 `kind` $\in$ {float, int, axis_range (值=(min,max,points), GUI 出三框), bool, choice}。 **任何值都不被 eval**——消费方按 `kind` 校验/强转。

MeasurementSpec(name, params, result_labels, x_key, y_key, build, grid_shape, metadata)

一个具名测量 + 它的参数声明 + 一个 `build(**param\_values)  $\rightarrow$  ScannedMeasurement` 闭包 (返回**未跑**的测量)。`x\_key/y\_key` 是发布到 hub 的信号名; `grid\_shape` 给逐站点测量重排成 2D 图用; `metadata` 携带额外信息 (如温度拟合需要的 `capture\_radius` 换算) 而不扩签名。

12.4 自动发现的测量目录: 开放注册表

`ReadoutSubsystem.measurement\_specs()` 返回 GUI 能驱动的测量目录, 由开放注册表 `operations/measurement\_registry.py` **自动发现**装配:`operations/measurements/` 包里**每个 @measurement 装饰的工厂** + notebook 里 `register\_measurement(...)` 注册的临时工厂。一个测量工厂是 `build(readout)  $\rightarrow$  MeasurementSpec`, 接收 readout 子系统 (于是 spec 的 `build` 闭包能捕获 session、复用子系统建器)。`discovered\_specs(readout)` 首次调用时 `pkgutil` 把整个包逐模块导入一遍 (每个 @measurement 随之注册), 之后用 readout 现造每个 spec, 按 `order` (内置在前) 再注册排序、按 `name` 去重 (后注册覆盖)。

加一个新测量 = 往 operations/measurements/ 丢一个 @measurement 文件(或 notebook 里 `na.register\_measurement(...)`), 它就**自动出现在 Add Panel 列表里**可被选中连接——没有任何“目录 list”要改, 内置的温度/读出时长两个 spec 也住在这个包里、与新测量平权。下划线开头的模块 (如 `\_helpers.py`) 被发现机制跳过, 可放包内私有助手。

12.4.1 单一真相源建器:build_*_scan

每个内置 spec 的 `build` 闭包**都路由到与 API 同一个建器**: `build\_temperature\_scan(...)` / `build\_detection\_scan(...)`。于是**API 一行 (`exp.readout.temperature(...)`) 与 GUI 的 Start 按钮组装的是同一个 ScannedMeasurement**——这是“单一真相源”在测量层的落地: `temperature()` 内部也调 `build\_temperature\_scan` 再 `run`, GUI 经 `spec.build` 调它, 两条路不可能漂移。工厂收到 `readout` 后用 `readout.session` 捕获 `exp`, 所以**console 端无需持有 session** (解耦: console 只拿到一摞已绑定的 spec)。用户自写 spec 同理:`build` 返回的对象只需有 `.axis.values + .measure(value, index)` (即一个 `ScannedMeasurement`), `ScannedMeasurementNode` 就能把它逐点驱动成 1d 曲线。

12.5 GUI 段: 自动参数表单 + 一键 Start

一个扫描 measurement 在 Add Panel 选中后是 Logic tab 上一个默认停的逻辑节点行; 它自己的 Edit (LogicNodeEditor, 绑这一个 spec, 无类型选择器——spec 已选定):

1. 由该 spec 的 params 自动生成带校验的表单: float/int → 带 lo/hi/unit 的 spin; axis_range → min/max/points 三框; bool → 复选; choice → combo。required 缺失时禁用 Start 并高亮 (带 *)。绝不自由文本 eval (confocal 教训)。
2. Start (一键): 按 kind 收集 + 校验参数 → `spec.build(**values)` 得未跑测量 → 包成 `ScannedMeasurementNode (x\_key/y\_key/grid\_shape 来自 spec)` 登记进 `console.running\_nodes → node.start(rate\_hz=...)` 后台跑。节点只 **publish 到 hub、不自动出图** (plot=False); 你自己在 Monitor 加一张 1D Plot 面板把 Source 指 `value = <y\_key> vs <x\_key>` 看它逐点长出。Stop → `node.stop()` (协作取消)。
3. `\_poll\_logic\_nodes` 每 tick 检查节点完成跃迁 (即使版本不变也能捕捉自停), 完成后释放控件、行点回灰 (温度 spec 的结果还可由 plot 面板 Edit 的 Fit 跑 `fit\_temperature`)。

12.6 虚拟 == 实机

整个引擎只碰 `camera.acquire / sequencer (经 pulse) / calibration.signals|detect / active\_plotter().run`; 零 `import virtual/qcmos`、零读仿真真值。放在 `operations/` 自动受 `tests/test\_virtual\_equals\_real\_contract.py` 守护。换真机: `na.connect("qcmos", ...)` 后, 同一个 spec、同一个 Start 按钮、同一条 `exp.readout.temperature(...)` 一行不改地跑。

12.7 1D 面板的 (N,2) x-y 渲染

扫描结果的 x 是真实物理量 (`t_off` 秒、detection time 秒), 不是 0,1,2 index。为此 1D 面板 (Live1D) 支持 (N,2) 数组当 x-y 曲线 (`col0=x, col1=y`): `ScannedMeasurementNode` publish 的累积 x/y 被 1D 面板的 source 组装成 (N,2) 喂进来, 于是横轴刻度是真实 `t_off`。

Note

已知取舍: (N,2) x-y 是纯形状启发式 (`ndim==2 & shape[1]==2`)。普通 1D 面板从不画两序列, 碰撞面有限, 但理论上某表达式恰好算出 (N,2) 会被静默当 x-y。更稳的做法是只在“结果卡” (`config.params` 带 `measurement`) 才走 x-y 分支——已 flag 给后续。

13

一键温度:release-recapture 测温

release-recapture (弹道重捕) 测温是通用 Measurement 框架里第一个“非保真度”测量，**一键启动**：扫 trap-off 时间、逐点测存活、拟合出温度。

13.1 物理: 弹道重捕

两次成像之间把 trap**关掉** `t\_off`，原子按热速度自由飞；再开 trap，问还有多少原子近到能被重捕。越热的原子热速度展宽越大、飞得越远，存活随 `t\_off` 衰减越快——**survival-vs-t_off 曲线就编码了温度**。模型 (`release\_recapture\_survival`, 3D 各向同性点源、忽略初始位置展宽的短时紧附近似)：每轴重捕概率是高斯速度分布在 $|v| < r_c/t$ 窗内的比例 (误差函数)，各向同性存活 $= \text{baseline} \cdot P_{\text{axis}}(t)^3$ ，其中 1D 速度展宽 $\sigma_v = \sqrt{k_B T/m}$ 。误差函数复用共享的 `normal\_cdf` (不另抄一份)。

Note

capture_radius 简并 (关键)：曲线只定 $r_c/\sqrt{T}$ ，所以**必须**从阱几何提供捕获半径 `capture\_radius` (米；推子阱约等于束腰) 才能定出 T。因此 `fit\_temperature` 把 `capture\_radius` 当已知输入、只拟合 T (与可选 baseline)。

13.2 序列:6 周期双触发,t_off=duration scan slot

`build\_release\_recapture\_pulse` 建一个 6 周期 `PulseTableState`，**两个相机触发**夹一段 trap-off：

text

#	period	trap	probe	emCCD	duration
0	image1_expose	1	1	1	exposure (第 1 个触发上升沿)
1	image1_settle	1	0	0	settle
2	trap_off	0	0	0	t_off <- 绑定到 duration ↪ scan slot s0
3	trap_recapture	1	0	0	recapture
4	image2_expose	1	1	1	exposure (第 2 个触发上升沿)
5	image2_settle	1	0	0	settle

周期 2 的 duration 经 `bind\_field("duration", "2")` 绑到 scan slot `s0`——与扫描读出时长完全同一种可扫量, 所以 `pulse.set\_time` 逐点驱动 `t\_off`, 硬件还能流式扫一整张 `t\_off` 表。为什么不能“重复单帧序列”: 两帧必须同一次装载、中间夹 `trap=0`; `finite\_frame\_sequence` 会在帧间重装原子, 两帧就成了无 `trap-off` 的两次不同装载, 存活无意义。 `ReleaseRecapturePlan` 因此固定 `n\_frames=2`: 一次 `acquire` 取回 `[image1, image2]`。

`SurvivalReducer` 用 `calibration.detect` 出两帧占据 `occ1/occ2`, “存活”= 两帧都占据; 标量形式回初始占据站点中的存活比例 `(occ1 & occ2).sum / max(occ1.sum, 1)`, `per\_site=True` 回逐站存活向量 (占据处 1/0, 未占据处 nan)。

13.3 GUI 路径 (一键)

Add Panel → 选 Measurement: Temperature → Add Panel (在 Logic tab 建一个停着的扫描节点并打开它自己的 Edit 标签) → 在该节点 Edit 的表单填 Trap-off time (min/max/points, us)、Shots/point、Capture radius (um, 必填带 *)、Per-site survival (可选) → Start。节点按测量 slug 把信号发到 hub (`temperature\_t\_off / temperature\_survival`); 你在 Monitor 加一张 1D Plot 面板, Setting 里 signal 选 `temperature\_survival` (Source 即 `value = signal`), 就看到 survival-vs-t_off 曲线逐点长出——x 轴标签/单位 (Trap-off time (s)) 与 x 值由同一个产出节点自动解析, 只需选一个信号。要拟合 $T \approx \dots \mu\text{K}$, 在那张 plot 面板 Edit 的 Fit 段跑。spec 内置默认 RR 序列 (用当前 sequencer 的 channels 建), 无需手动绑脉冲。

13.4 API 路径 (同源)

Python

```
import numpy as np
import Zou_lab_control.neutral_atom as na

exp = na.connect("virtual", sitemap={"grid_shape": (5, 7)})
exp.readout.sitemap(method="box"); exp.readout.thresholds()

state = na.build_release_recapture_pulse(channels=list(exp.devices.sequen_
↪ cer.channels))
pulse = na.bind_pulse(exp.devices.sequencer, state)

res = exp.readout.temperature(np.linspace(0, 300e-6, 13), pulse=pulse,
↪ shots=16)
fit = na.fit_temperature(res.x, res.y, capture_radius=6e-6)
print(fit.summary())      # {'temperature_uK': ~44..50, 'capture_radius_m':
↪ 6e-06, ...}
```

GUI 的 Start 调的就是同一个 `build\_temperature\_scan` (spec.build 包它) → API 与 GUI 不可能漂移。 `fit\_temperature` 是纯后处理 (在完成的 `ScanResult` 上跑, 绝不进 measure 循环——让 live 引擎不沾物理模型)。

13.5 虚拟 trap-off 损失模型 (只在数据源侧)

虚拟相机后端在数据源侧建模释放-重捕损失: 从 fired 的 `PulseSequence` 解析 trap-off 时长, 用 `stdlib erf` 算重捕存活概率并按之随机丢原子 (`reload` once-per-shot 防跨 shot 复利)。这不污染分析层 (分析层只看相机帧、只走 `detect`): survival 在虚拟上 0..300μs 从 0.97 衰到 0.06, 拟合回约 44–52μK (注入 50μK)。符合“只 fake 最底层数据源”。

14

命名布局: 设计一次, 随处载回

布局整包存成一个 JSON, 文件名就是任务名。没有内置预设——唯一的“任务”就是你自己 Save 出来的布局。存进去的不只是几何, 而是面板、位置、尺寸、表达式、每个 Edit 表单的参数值 (logic 节点值 + plot 参数) + Logic 节点声明。三个入口完全等价:

- GUI: 标题栏 Save 默认存到 `tasks/< 任务名 >.json` (任务名即左上的名字框); Load 选一个 JSON 整套载回。Save* 黄星 = 有未保存改动。
- 命令行: `task\_console.bat -task < 你存的名字 >` (读 `tasks/< 名字 >.json` 或一个 JSON 路径)。
- Notebook: `zf.show\_task\_console(hub=hub, task="< 你存的名字 >")`。

配一套新监控台的完整流程: 开一个空控制台 → 从 Add Panel 加 Logic 节点 (相机 measurement / 扫描测量 / processor / task) 并 Start、加 Plot 面板把 Source 指到它们发布的信号 → 改任务名 → Save。以后一条命令整套载回——实验室里每类日常实验配一套自己的监控台。

14.1 状态序列化

`TaskConsoleState: name/interval\_ms/panels[]/logic[]`; 每个 `PanelConfig` 含 `kind/title/row/col/size/source/params` (params 含该面板 Edit 的参数值), 每个 logic 节点声明含 `kind/name` + 它 Edit 表单的参数值, `to\_dict/from\_dict` 往返。布局 JSON 不含任何几何像素 (几何归 frontend), 所以跨机器可移植。

15

空白收紧

`PANEL\_MARGINS\_PX (live.py) = (110, 86, 80, 70)` (左右下上), 所有 kind 共用同一套——这是**少留白**与**不裁切**的平衡:

- 左 110 (= `STOCK\_MARGINS\_PX[0]`, confocal 左边距) 是容纳 4–5 位 y 刻度标签 (如 qCMOS ROI 像素 1180) + 旋转 y 轴标题的**最小值**; 再窄, y 轴标题就会被挤出 figure 左缘 (每种 kind 都如此)。这一侧不能为了省空白而收窄。
- 右 86、下 80 相对 confocal 的 110/100 收紧, 使数据区占 figure 面积约 50%, 卡片不留多余横向空白, 且 footer **紧贴 canvas** (不靠 `addStretch` 撑高)。
- 顶 70 = `TITLE\_SLOT\_PX` 的 title 槽, 常备。 `\_with\_title\_margin` 的下限与它读**同一常数**, 故有无标题的面板一样高、 `panel\_display\_size` 不会少算卡片高度。

三档 DPR 真实入口截图验过不裁切、不重叠、对齐; 现有 5 类 plot 的几何只在 panel 路径上按预期变化。收紧让 agg buffer 变小, 也顺带降一点重绘成本 (见性能章)。

Note

结构性残留 (已 flag、未强改): 右侧仍有一段 gutter, 多行卡片 tiling 也有 slack。这是**固定拖拽网格 + 密封几何的结构性权衡**——用户喜欢的拖拽-吸附网格按整数布局槽排, 卡片外框必须吸收图边距的模数差额, 于是窗口宽度不被面板占满时出现右 gutter。三档 DPR 表现一致, 非回归。要消掉需改成“面板默认占满窗口宽度”或自由布局, 是更大的取舍, 留给用户拍板。

16

性能结论 (诚实版)

诚实结论先行：多 live 同时刷新的卡顿瓶颈是密封 300dpi 下 matplotlib 对约 10 个子图的文字栅格化 (每帧重画, **intrinsic**)。两条主流提速方案**都实测否决**；合规的提速来自空白收紧；更激进的提速需要用户拍板是否接受可见行为/外观变化。源数据见 `\_redesign/03b\_perf\_conclusion.md`。

16.1 量化 (demo_console=5 面板, 离屏, 40 tick)

- **compute/tick ≈ 13 ms** (`\_tick` $\rightarrow$ 各卡 `update\_core`: 直方图 + 2 个 `curve\_fit` + `numpy`) ——GUI 线程同步阻塞部分, **很小**。
- **draw/tick $\approx 72\text{--}75$ ms** (5 面板 full agg 重绘)。“steady-draw (不喂新数据) = 72 ms $\approx$ (有数据) - (无数据)” $\rightarrow$ **重绘成本与 limit 是否变化无关, 是 intrinsic**。
- cProfile 最大头: `draw\_text` (跨约 10 个轴 @300dpi 的字形栅格化), 与 `margin/limit/能否 blit` 都无关; 其次是每 draw 都跑的位置重解算 (`\_update\_title\_position/get\_tightbbox/tick boxes`)。

16.2 评估过的方案 (各附一句根因)

- **blit 增量重绘——否决 (有硬数据)**：实测 **152 ms/tick, 比 full draw 还慢 60%**。根因：(a) live 数据持续 autoscale, `limit/tick` 几乎每 tick 变 $\rightarrow$ fast-path 基本不触发; (b) 300dpi figure buffer 巨大, `copy\_from\_bbox/blit` 本身每帧约 11 ms/panel 纯开销。blit 只在 limit 稳定时才赢, 活体 autoscale 破坏了前提。
- **冻结 title/xlabel 位置——否决 (违反 DPR 中性)**：draw -19% (72 $\rightarrow$ 58 ms), 但 **sf1.5 有 4845 像素翻转** (freeze-vs-freeze 自比 0 差 $\rightarrow$ 确证是 freeze 引入)。根因：`show()` 首绘在最终 dpr 之前冻结了 dpr 错误的位置, sf1.5 下 rounding 翻转。违反“三档 DPR 都要中性 (DPR=1 不算过)”。
- **降 dpi / 减 tick / 减轴——否决**：违反 frontend 密封视觉契约 (300dpi 单一字体源)。

16.3 合规提速 + 留给用户的开关

已落地的合规提速 = 空白收紧（上一章）：figure 像素面积下降 → agg buffer 与面积成正比的 fill/resample/draw_path/copy-to-Qt 那约 30–40% 变快（draw_text 本身不变）。即空白修复同时是**合规的提速**，且正是用户想要的视觉改进。

更激进的提速会**改变可见行为或视觉**，**不擅自做，标记给用户拍板**：迟滞 / sticky autoscale（让 limit 稳定从而 blit 生效）、错峰重绘（不每 tick 重画全部面板）、嵌入面板低 dpi 性能模式。三者都能显著提速但改 limit 跟踪 / 刷新语义 / 视觉，不在“不改外观不改逻辑”内。

17

frontend 密封 API 契约

frontend 对外暴露一个**小而密封**的接口: 外部只传数据 + 少量语义选项, 拿回正确的美术/几何/dpi/字体, **绝不能通过任何公共接口搞乱视觉设计**。权威定义在 `frontend/\_\_init\_\_.py` 模块文档 + `frontend/AGENTS.md` + `docs/MAINTAINER\_NOTES.md` §18。六条规则:

1. 几何被拥有、永不外传: `plot()/panel\_plot()` 拒绝 `spec/data\_px/margins\_px/dpi/bad\_color/colors` (内部几何走私有 `plot(\_spec=)`)。
2. 尺寸/cmap 是 curated 且入口校验 (`PANEL\_SIZES/panel\_size\_cells`); pulse 几何常量私有化。
3. 一套排版一个 dpi: `style.DEFAULT\_STYLE` 是只读 `MappingProxyType`。
4. 一条缩放规则: `resolve\_fluent\_auto\_scale(GUI)+panel\_canvas/PANEL\_DISPLAY\_SCALE` (嵌入图); `show\_*` 默认 `scale=None`。
5. 带美术的 fluent 控件保持内部: `qt\_fluent.* / qt\_canvas.*` 不进 `\_\_all\_\_` (阴影/边框/圆角是构造细节)。
6. 加公共参数前先分类: `DATA(labels/title/bins/thresholds/relim/cmap-from-list/channels)` 允许; `ART/GEOMETRY/TYPOGRAPHY` 只能在内部类上。

Task 控制台严格遵守: 面板只通过 `panel\_plot+size` 预设作图, 拟合只经 `DataFigure(card.plotter)`, 改图只改 `config` + 重建——从不戳 figure inches/边距。

18

frontend 绘图组件参考

18.1 工厂与 live 类

`plot(data\_x, data\_y, *, kind, update, labels, display, data\_figure, ...)` 是统一工厂 (数组契约 `data\_x=(N,coord\_dim)/data\_y=(N,channel\_dim)`); `panel\_plot(data, *, kind, size)` 是它的仪表盘特化 (不 `display`、不附 `DataFigure`, 由宿主嵌 `canvas`)。五个 live 类: `Live1D/Live2DDis/LiveLiveDis/HistogramFigure/LiveSiteMap`, 都公共导出、可直接构造, 但首选 `plot()`。

18.2 LiveSiteMap

原子阵列站点图: `data\_x` 是 $(N,2)$ 站点中心 (相机像素), `data\_y` 是逐站值; 每站一个 `EllipseCollection` 实心圆按值着色, 可选 `image` 垫相机帧底图、`cmap`、`roi\_radius`。同 2D 的 `[0.75,0.1,0.1]` 分割, `square` 主图与 `colorbar` 逐像素对齐——融入既有几何而非另起炉灶。

18.3 DataFigure(拟合/存图/单位)

`DataFigure(live\_plot)` 从一个 live 绘图对象建数据句柄。关键方法: `lorent/gaussian/decay/rabi/lorent\_zeeman/...` 接 `p0=/is\_fit=/*kwarg`s 固定命名参数)、`xlim/ylim`、`change\_unit`(单位环)、`clear`、`save(path)`(写 `png+npz`、时间戳、返回 `{figure,data}`)。每面板 `Edit` 标签的拟合/存图全部复用它。

18.4 新增一种 plot type 的方法 (以 sites 为模板)

1. 在 `live.py` 写一个 `BaseLivePlot` 子类, 几何必须走 `panel\_plot\_spec` 的共享区域 (用 `split\_axes\_horizontally` 等), 并进 `plot()` 工厂的 `kind` 分发 + `\_normalize\_kind` 别名。
2. 在 `task\_console.py` 的 `PANEL\_KINDS/\_DEFAULT\_SOURCES/PANEL\_PARAMS` 各加一项; `\_coerce/\_build\_plot/\_render` 各加一个分支。
3. 若需要辅助信号 (如 `sites` 的 `centers/image`), 在 `\_xxx\_aux(namespace)` 里取并校验, 错误进该面板状态行。

-
4. 可视化改动必须用 `devtools.capture\_user\_view` 三档缩放截图验收 (`parity` 目标对比两 GUI 控件尺寸)。

19

扩展到真实设备

这是你最关心的部分: 虚拟仪表盘怎么变成真机监控台。核心结论: GUI 一行不用改, 只需把虚拟相机换成真机相机 (只换 `connect()`)、按约定信号名 `publish`。

19.1 最小接线 (notebook, 自管循环)

Python

```
from Zou_lab_control import frontend as zf
from Zou_lab_control.neutral_atom.core.signals import SignalHub
import Zou_lab_control.neutral_atom as na

hub = SignalHub()
console = zf.show_task_console(hub=hub) # 开局空; 自己加 Plot 面板把 Source
↳ 指到下面 publish 的信号

exp = na.connect("virtual") # 真机换成对应 config
centers = exp.readout.sitemap(frames=12,
↳ display=False).calibration.centers
rate = None
for shot in range(100000):
    frame = exp.camera.acquire(1, sequence=exp.sequence,
                               sequencer=exp.devices.sequencer)[0]
    det = exp.readout.from_image(frame, display=False) # AtomDetection
    occ = det.occupied.astype(float)
    rate = occ.mean() if rate is None else 0.95*rate + 0.05*occ.mean()
    hub.publish({"frame": frame, "counts": det.counts, "occupied": occ,
                "centers": centers, "rate": rate, "shot": float(shot)})
```

要点: □ 沿用约定信号名, 同一套命名布局在虚拟/真机间零修改通用; □ `centers` 来自一次 `sitemap` 标定 (`TrapCalibration.centers`), 让 Site map 面板有站点坐标; □ `readout.from_image` 走的就是逐帧 detect 用的同一套检测原语, 所以行为一致。

19.2 用内置逻辑节点拼装载读出

更干净的做法是用内置的三类逻辑节点 (`operations/logic.py`) **自己拼**——每个节点只经 `CameraDevice` 契约取帧、用与真机相同的检测原语, 换真机**只换 connect()**:

Python

```
from Zou_lab_control.neutral_atom.operations.logic import (
    CameraMeasurement, OccupancyProcessor, CalibrateReadoutTask)

# 1) 先跑标定 task: 产 TrapCalibration (站点中心 + 逐站阈值)
cal = CalibrateReadoutTask(hub, exp.camera,
    ↪ sequencer=exp.devices.sequencer,
    ↪ grid_shape=(5, 7),
    ↪ mode="box").run_to_completion()

# 2) 相机 measurement: 只发 frame
cam = CameraMeasurement(hub, exp.camera, sequencer=exp.devices.sequencer)

# 3) detect processor: 逐帧 frame -> occupied/counts/rate (跑真
    ↪ calibration.detect)
detect = OccupancyProcessor(hub, calibration=cal.calibration,
    ↪ grid_shape=(5, 7))

cam.start(rate_hz=5.0); detect.start(rate_hz=5.0)
console = zf.show_task_console(hub=hub, running_nodes=[cam, detect])
```

`running\_nodes=[...]` 把节点登记到控制台并在开窗时自动 `start` (每节点自己 Logic Edit 的 Start/Stop 管理它的生命周期)。没有**一键复合体** (没有 `build\_loading\_readout`、没有“Readout: Loading”复合节点)——三个节点独立、信号显式相连。GUI 里同样: `Measurement: Camera (live frames)` + `Processor: Judge occupancy` + `Task: Calibrate readout (Judge occupancy 的 calibration 留空即用会话标定, 或填 calibrate task 存出的路径)`。**线程安全**: `shot()` 只产数据、`publish` 负责拷贝, GUI 在另一线程读副本——节点可放心复用缓冲。阻塞式硬件采集放在节点线程, 绝不放 UI 线程。

19.3 对接 references 里的 Rb87 qCMOS 读出

`references/.../rb87\_readout\_v16` 是一套真实 qCMOS 读出流程: 4-shot group (20ms 参考 $\times 3$ + 短曝光读出)、模板找站点 (`find\_sites\_from\_template`, 35 站)、PSF 加权提取标量信号 (`box/psf/weighted-logpsf`)、20ms 参考双高斯定阈值、短曝光 per-site 阈值训练出保真度 `F\_bal`、单帧 `single\_predict` 出占据。映射到本仪表盘:

Rb87 读出产出	仪表盘信号 / 面板
短曝光帧	<code>frame</code> → 2D image
<code>single_predict</code> 占据 (35,)	<code>occupied</code> → Site map(占据)/1D
per-site 信号 (photons)	<code>counts</code> → Distribution(双高斯 + 阈值)
站点格点中心 (35,2)	<code>centers</code> → Site map 站点坐标
装载率 = <code>mean(occupied)</code>	<code>rate</code> → Rolling trace
逐站 <code>F_bal</code> / 阈值	<code>fidelity_sites / thresholds</code> → 1D / Site map

也就是说: 把 Rb87 读出循环的每个 group 算完后 `hub.publish` 这些量, 你自己拼好的布局就直接显示; 逐站读出保真度这类离线刻画交给目录里的一次性 `Readout fidelity processor` (在一个已存文件

夹上跑 `characterize\_from\_dir`、发 `fidelity\_site/fidelity\_threshold` + 标量汇总, 可写回会话标定阈值)。 `rb\_plots.py` 里现有的画图 (per-site 分布网格、参考双高斯、ablation 曲线) 中, 分布/站点图/滚动量都被现有面板覆盖; **时序漂移**、**per-site 保真度条**、**ablation 曲线**是目前面板覆盖不到、值得新增的类型 (见下章待决方向)。

19.4 真机 vs 虚拟的唯一差别

虚拟相机用 `VirtualTrapArray` 渲染, 真机用相机; 两者都经同一 `CameraDevice` 契约、都走 `find\_site\_centers/roi\_counts/Otsu`。所以:**在虚拟上设计好的布局、验证过的面板/拟合, 接真机时不需要改任何 GUI 代码**, 只换相机数据源。这正是五层解耦的回报。

20

测试与验收

- 契约/渲染:hub 契约、逻辑节点信号 + 收敛、六种面板渲染 + 更新 (含两种 live 类型)、A-B 表达式 + 错误隔离、布局 JSON 往返、增删 + 脏星 + 存盘、命名布局往返、信号下拉、Site map 坏 centers 隔离。
- 节点契约:OccupancyProcessor 的 `occupied = cal.detect(frame).occupied`、task 向 hub publish 0 信号且结果留实例、calibrate task 三种 mode + plot 分流 (`tests/test\_task\_cali\_modes\_and\_plot\_split.py`)。
- 几何/缩放:confocal 模数契约 (签名 =["size"]、dpi、margins、PANEL_SIZES)、卡片 tiling(`2x(1x2)+gap=2x2`)、EmbeddedFigureCanvas 三档缩放不变量、两 GUI auto-scale parity。
- 密封 API:`plot/panel\_plot` 拒绝几何 kwargs、`DEFAULT\_STYLE` 只读、`pulse\_plot\_spec` 无 margins/dpi。
- Setting 瘦身契约:Setting 只含基本控件、**无 Fit/无 relim** (扫 `\_build\_settings` 产出的控件集; 含 `not hasattr(card, "fit\_combo")` 等断言)。
- **通用 Measurement**:spec 参数 round-trip、ScannedMeasurementNode 走契约不读真值且自停、虚拟 e2e 出曲线 (survival 单调衰减 + 拟合 T)、SurvivalReducer 正确性; 扩进 `test\_virtual\_equals\_real\_contract` (measurement/temperature 不 import virtual/qcmos、不读 ground-truth)。
- Processing: 每面板 Edit 标签绑定快照、命令拟合叠加 (全套模型含 2D center)、Apply lim、Save Fig 产 png+npz、Clear。
- PDF 接口:`render\_tex\_pdf` 临时目录只产 pdf、`render\_notes\_pdf` docs 只留 pdf。
- **可视化验收强制规则**:任何 UI/plot 改动必须 `capture\_user\_view` 三档 QT_SCALE_FACTOR(1.0/1.25/1.5) 整窗截图 + 1:1 裁剪;DPR=1 离屏通过不算通过。

21

公共接口与内部结构索引

21.1 对外公共 (notebook/实验室会调)

zf.show_task_console(*, hub, state=None, task=None, running_nodes=(), measurements=(), processors=(), tasks=())
开控制台窗口; measurements/processors/tasks 三份目录各列进 Add Panel 对应组;
running_nodes 预挂逻辑节点开窗时自动 start。

zf.panel_plot(data_x, data_y=None, *, kind, size="2x2", **kwargs)
仪表盘面板工厂。

zf.plot(...) 通用绘图工厂 (notebook)。

SignalHub / publish/latest/history/names/snapshot_latest 数据契约。

LogicNode / CameraMeasurement / ScannedMeasurementNode / Processor / OccupancyProcessor / Task / CameraMeasurement
逻辑节点基类 + 三类生产节点 (operations/logic.py)。

exp.readout.measurement_specs() / processor_specs() / task_specs() GUI
可驱动的三份目录 (传给 show_task_console(measurements=, processors=, tasks=)) ——**自动发现**, 非写死 list。

na.measurement / na.processor / na.task (+ register_*/ unregister_*) 把一个 build(readout) → *Spec 工厂注册进对应目录 (装饰器 / 函数 / 撤销); 或直接把 @measurement/@processor/@task 文件丢进 operations/measurements\,|\,processors\,|\,tasks/ 自动发现。na.axis_range_tuple 把 axis_range 参数值解释成 (min,max,points)。

21.2 task_console.py 内部要点

PANEL_KINDS/PANEL_PARAMS/ParamSpec/FIT_MODELS/COLORBAR_KINDS;resolve_task_state/default_task_state
几何_grid_pitch/_card_size/_pos_to_slot/_resolve_collisions;PanelCard(_build_setting)

22

已知限制与待决方向

下面是**留给你决定**的方向（按我判断的价值排序），每条注明现状与代价。

1. **逻辑节点 + 通用测量 + 一键温度（现状基线）**。三类逻辑节点（`CameraMeasurement` / `OccupancyProcessor` / `CalibrateReadoutTask`，走相机契约、自标定）、通用 Measurement 框架（`MeasurementSpec` / 三角色 / `ScannedMeasurementNode`）与 `release-recapture` 一键温度都在库里；换真机 `na.connect("qcmos", ...)` 走同一路径。下面的方向在此基线上展开。
2. **性能激进档需用户拍板**。瓶颈 = 密封 300dpi 文字栅格化 (intrinsic), blit / 位置冻结已实测否决。迟滞 autoscale / 错峰重绘 / 嵌入面板低 dpi 性能模式能进一步提速, 但都改可见行为或视觉, 需要你决定是否接受 (见性能结论章)。
3. **(N,2) x-y 形状启发式收口**。扫描曲线靠“列数 == 2”判 x-y; 建议改为只在面板 source 明确引用某扫描节点的 `x\_key/y\_key` 时才走 x-y 分支, 避免普通 1D 面板碰巧产 (N,2) 被误判。
4. **死码/残留清理 (minor)**。`live.py` 的 `\_set\_colorbar\_ticks` 自递归 no-op (2D 靠 `update\_core` 内联设刻度、sites 靠 matplotlib auto-tick, “修”它会改 sites 基线 → 删或显式注释 no-op, 走零基线漂移); `PanelCard.\_fit\_df` / `FITTABLE\_KINDS` 随 Fit 迁走后只剩测试消费, 可干净删除。
5. **时序/漂移面板**。现有 monitor 是单标量滚动; Rb87 的“站点中心随 group 漂移”逐站 F_bal 条形“drop-worst-sites ablation 曲线”是新面板类型 (多线/条形/scatter)。要不要加, 取决于你日常盯哪些量。
6. **Processing 是否要直接编辑 live 图**。目前是**快照**(冻结、独立)。若想在 Edit 里直接调 Monitor 卡的 clim/范围并实时反映, 需要 reparent live canvas 或双向同步, 复杂度上一个台阶。倾向保持快照模型 (简单、安全)。
7. **控制台 ↔ 脉冲 GUI 联动**。同一进程里共享 sequencer, 在控制台里直接触发 `on_pulse` / 改 `detection time` 并看结果——把“配置脉冲 → 采集 → 监控”闭环到一个窗口。这是较大的一步, 需要想清交互。
8. **多 group / 4-shot 结构**。Rb87 是 4-shot group; 若要在仪表盘层面理解 group, 需要 hub/节点支持 group 语义。多数情况下“循环里算完再 publish”已足够, 不必上 group 抽象。

22.1 优先级

(1) 是现状基线。其上的优先级:(2) 性能激进档需先定是否接受行为/视觉变化、(3)(4) 形状启发式收口与死码清理较顺手、(5)(6)(7) 是更大的架构决定, 定方向后再做。